How Hard Is It to Factor Homogeneous Polynomials?

ScottLean4 · for Dana Scott

The answer is a pleasant surprise: *computationally, it is easy—polynomial time.* This is worth savoring, because the sibling problem, *integer* factorization, is believed hard (no polynomial algorithm is known, and much of public-key cryptography rests on that). Polynomials factor in polynomial time; integers, apparently, do not.

Binary forms (two variables): as easy as univariate

A binary form $F(x, y)$ of degree d is equivalent to a one-variable polynomial. **Dehomogenize**—set $y = 1$ to get $f(x) = F(x, 1)$ of degree $\leq d$—factor that, then **re-homogenize** each factor (restoring any lost factor of y). So factoring binary forms is *equivalent* to univariate factorization:

- Over $\mathbb{Q}$: polynomial time, by **Lenstra–Lenstra–Lovász (LLL, 1982)**—lattice basis reduction made univariate rational factoring provably efficient.
- Over a finite field $\mathbb{F}_q$: polynomial time (randomized), via **Berlekamp** and **Cantor–Zassenhaus**.

Geometrically, a binary form of degree d is exactly d points on the projective line $\mathbb{P}^1$ (its roots with multiplicity). Over an algebraically closed field it splits completely into d linear factors. The degenerate conic

$$x^2 - y^2 = (x - y)(x + y)$$

is just "the binary quadratic has two roots on $\mathbb{P}^1$"—the pair of lines $x = y$ and $x = -y$.

Ternary and higher forms: still polynomial time

A homogeneous polynomial in n variables dehomogenizes to a general polynomial in $n-1$ variables. Multivariate factorization over $\mathbb{Q}$ or $\mathbb{F}_q$ is *also* polynomial time—**Kaltofen (1980s)**—by evaluating to the univariate case and then **Hensel lifting** to recover the multivariate factors. Homogeneity is a mild convenience (each factor is itself homogeneous), not a change in complexity class.

Where the difficulty actually lives

The hardness is in the constants and the field, not the complexity class.

1. **The base field decides the answer's shape.** Over $\mathbb{C}$ everything splits into linear forms; over $\mathbb{R}$ one gets linear and irreducible quadratic forms; over $\mathbb{Q}$ higher-degree irreducibles persist and factor coefficients may live in extensions. Same algorithm, very different output.
2. **Coefficient growth.** LLL and Hensel lifting are polynomial, but intermediate coefficient heights can swell, so a large-height form is slow in practice though poly-time in theory.

3. **Verify versus discover.** *Verifying* a factorization is trivial (multiply out). *Discovering* it is the algorithmic work above. Both are easy in the complexity sense; only the second is interesting.

The formalization angle

So the *algorithm* is easy—but a *certified* factoring procedure is a different matter. Mathlib has the *theory* (polynomials over a field form a unique factorization domain, irreducibility, and so on), but there is no standard *tactic* that returns the factors *with a kernel-checked proof* the way `ring` verifies a guess. That gap is engineering, not complexity: producing the factors is fast; producing them together with a formal certificate is the work no one has fully packaged.